

Activity 3.4a

Comprehensive BST Activity

Karol Daniel Orozco Medina - A01647837

Omar Del Toro Covarrubias - A01648573

Timoteo Aguilar Bauermann - A01648827

Kevin Alexander Ramírez Mendez - A00844255

Programming of Data Structures and Fundamental Algorithms

Obed Noé Sámano Abonce

Grupo 601

Tecnológico de Monterrey Campus Guadalajara

31/10/2026

Why we chose a splay tree:

For implementing this we chose a splay tree ordered by the IP address. Each IPRecord stores an IP, the accesses (successful or failed) and for this we consider that an access is successful when the first word of the message is Accepted. We insert new IPs and whenever we access an existing IP the node moves to the root. This is a type of BST so the IPs always stay sorted.

The splay tree meets the monitor's three conditions. (1) It is built one line at a time and never needs the complete file. (2) Recording a line is O(log n) amortized, so it stays fast with thousands of IPs. (3) The amortized holds for any arrival order, because the tree restructures itself on every access. Also, in an attack the same IPs appear again and again, so these frequently used IPs stay near the root and their updates are cheap.

How the code works:

The program loads the file into a vector and updates the tree one record at a time, it counts an access as a success if the message starts with the word Accepted, anything else counts as a failure, if the IP is new we create a node and splay it to the root, and if the IP already exists, we splay the node to the root so that the IP never is repeated in two nodes. The logs must be sorted by date before running the monitor so that each month produces only one report. To make the report, the monitor traverses the tree inorder to collect the records and then it orders them using merge sort.

About the implementation:

1. Which structures from the course did you consider and discard? For each one, explain which of the monitor's conditions it does not meet.

For implementing this we chose a splay tree ordered by the IP address. We evaluated each structure against the monitor's three conditions: (1) it cannot stop and must work with data arriving continuously, (2) each new line must be recorded quickly even with thousands of IPs, and (3) it must stay fast regardless of arrival order. A splay tree is a type of BST so the IPs always stay sorted and each new line is recorded quickly because of the splaying function.

Structure	Reason for discarding it
Sorted Vector	Searching takes O(log n) but inserting a new IP moves O(n) records, so this makes the updates less efficient.
Normal BST	Searches and updates can take up to O(n) time.
AVL Tree	Meets the requirements of the monitor and guarantees O(log n) but we chose splay tree because if we repeat accesses to the same IP or close IPs it can be faster.

2. What is the complexity of each operation of your monitor (recording a line, querying an IP, querying a range, obtaining the IPs with the most accesses)? Distinguish the best and the worst case.

Operation	Best Case	Worst Case
Update the tree (one line)	O(1)	O(m) for one update O(log m) amortized
Search for one IP	O(1)	O(m) for one search O(log m) amortized

Get the top five or fifteen IPs	O(m log m)	O(m log m) sorts allthe records
Query an IP range	O(1)	O(m)

3. What is the least favorable arrival order for your structure, and what does your evidence from step 3 show? If your structure has a weak point, how would you solve it?

Ascending and descending arrivals are the least favorable because both produce a chain. Our tests compare insertion time and tree height for ascending, descending, and shuffled arrivals. With n distinct IPs, ordered arrivals produce a height of n − 1. This shows that the tree can become highly unbalanced. Ordered insertion itself can still be fast because each new IP is inserted near the root, but a later search can take up to O(n).

The weak point is that individual operations can take O(n), even though searches and updates take O(log n) amortized. One solution would be to periodically rebuild the tree into a balanced shape after each monthly report. We could collect the records using inorder traversal and rebuild the tree by choosing the middle record as each subtree's root. This would cost O(n) per rebuild but this will reduce the height to O(log n) immediately.

```
Monitor → use file 'bitacora_ordenada_date.txt'
Bitacora filename ('bitacora.txt'): bitacora_ordenada_date.txt

[1] Sort bitacora
[2] Search by date
[3] Search by ip address
[4] Monitor
[5] Monitor Test
[0] Exit
> 5

        [a] Ascending order
        [b] Descending order
        [c] Shuffled
> a
Number of registers: 90000
==== Ascending order ====
Splay Tree height: 89999
>> Time (Ascending): 10796 us

[1] Sort bitacora
[2] Search by date
[3] Search by ip address
[4] Monitor
[5] Monitor Test
[0] Exit
> 0
Quitting...
```

4. Your monitor obtains the IPs with the most accesses once a month. What would you have to change if the security team wanted that information updated after every line?

Sorting all the records after every line would cost O(n log n) per update. We would add a second balanced tree ordered by access count, using IPs to resolve the ties. Each update would remove the old ranking entry, increase the count, and insert the new entry.

About the investigation

1. From which month could the IPs that ended up with the most accesses be identified? What advantage does a live monitor have over analyzing the complete file at the end?

The IPs were already in the top five in June, the first month. For example, 203.0.113.140 had 194 accesses and finished with 940; 203.0.113.227 had 177 and finished with 855. The final order appeared in July and remained through October. A live monitor could have flagged these IPs in June, so the administrators could block them while the attack was still happening instead of finding out after the file was complete. It also shows how the counts grow month by month.

2. Look at the success percentage of your 15 IPs with the most accesses. Where does the suspicious group end and normal traffic begin? Is a high number of accesses enough to declare that an IP is malicious?

The first nine IPs had no successful accesses, while the next six had success rates between 92.10% and 96.28%, which looks like normal traffic from heavy users: many accesses, but most of them succeed. However, a high number of accesses is not enough to declare if an IP is malicious. We would also examine timing, targeted accounts, domains, and message details.

3. What information in each line of the log is your monitor not using? What question about the attack would you like to answer with it?

The monitor uses the IP, the month and the first message word to identify success. It doesn't analyze the day, time, port, domain or remaining message text. Log stores these fields but the IPRecords doesn't summarize them.

The question we would ask is: Were the 9 IPs with no successful accesses targeting the same accounts or trying different targets? If we compare attempts by time, domain, port and username we can know if they are targeting some particular accounts.